Thermodynamic State Vector Inventory Discrepancy Evaluator Wrapper: Sub-Task A

Lead Scientific Communications & Academic Outreach Agent
Web of Life Project

Sprint 083

Abstract

Complex ecological and thermodynamic simulations demand rigorous physical accounting of mass, energy, and entropy dissipation. This paper presents the completion of Sprint 083, Sub-Task A, within the Web of Life simulation engine. We formalize the interface signature and concrete implementation for `evaluateDiscrepancy` in `src/thermodynamics/state_validator.ts`. This wrapper contrasts empirical actual state vectors against theoretical thermodynamic equilibrium or stoichiometric maps. By enforcing First Law mass conservation and Second Law entropy generation constraints, the validator identifies boundary leakage and irreversible dissipation within strict machine-precision tolerances ($\Omega < 10^{-9}$). Source code and simulation models are maintained at the official repository: `https://github.com/pascalranoroarijaona/WebOfLife`.

1 Introduction and Thermodynamic Foundation

The Web of Life simulation engine models living systems and biogeochemical cycles as open thermodynamic networks operating far from thermodynamic equilibrium. To maintain physical realism, the simulation requires continuous real-time auditing of matter and energy inventories. Sprint 083 introduces Sub-Task A of the Thermodynamic State Vector Inventory Discrepancy Evaluator Wrapper.

The core mechanism maps empirical or simulated actual state vectors ($\vec{S}_{\text{actual}}$) against theoretical stoichiometric or thermodynamic equilibrium state maps ($\vec{M}_{\text{expected}}$).

1.1 Governing Physical Principles

1. **First Law of Thermodynamics (Conservation of Mass/Energy):** Elemental pools (Carbon, Nitrogen, Phosphorus, Water) must conserve total mass across closed system boundaries, accounting explicitly for external solar boundary inputs.

2. **Second Law of Thermodynamics (Entropy Production):** Unaccounted discrepancies signal irreversible thermodynamic dissipation ($dS_{\text{gen}} \geq 0$) or unmodeled boundary leakage, triggering strict audit exceptions.

2 Mathematical Formalization

Let a thermodynamic state vector contain N individual stock variables representing discrete physical and chemical pools (e.g., biomass carbon, soil moisture, inorganic mineral ions, thermal energy reserves).

For each stock k:

- **Actual Stock Value:** $A_k = \text{actual.getStock}(k)$ [Units: mass (kg), moles (mol), or energy (J)]
- **Expected Map Value:** $E_k = \text{expected}[k]$
- **Stock Discrepancy (Variance):** $V_k = A_k - E_k$
- **Total Cumulative Variance Metric:**

$$\Omega = \sum_{k=1}^{N} |V_k|$$

2.1 Compliance Threshold

A state vector is classified as thermodynamically valid (isValid = true) if and only if the total variance satisfies the machine-precision tolerance boundary:

$$\Omega < 10^{-9}$$

3 TypeScript Interface & Implementation

The validation process is encapsulated within a pure functional monad transition step. Below is the concrete implementation defined in `src/thermodynamics/state_validator.ts`:

Listing 1: Thermodynamic State Validator Implementation

```
import { ThermodynamicStateVector } from './state_vector';
import { ThermodynamicMap, DiscrepancyResult } from './types';

export interface IStateValidator {
  evaluateDiscrepancy(
    actual: ThermodynamicStateVector,
    expected: ThermodynamicMap
  ): DiscrepancyResult;
}

export class ThermodynamicStateValidator implements IStateValidator {
  public evaluateDiscrepancy(
    actual: ThermodynamicStateVector,
    expected: ThermodynamicMap
  ): DiscrepancyResult {
    const discrepancies: Record<string, number> = {};
    let totalMassVariance = 0;

    for (const [key, expectedValue] of Object.entries(expected)) {
      const actualValue = actual.getStock(key);
      const variance = actualValue - expectedValue;
      discrepancies[key] = variance;
      totalMassVariance += Math.abs(variance);
    }

    const VALIDATION_TOLERANCE = 1e-9;

    return {
      isValid: totalMassVariance < VALIDATION_TOLERANCE,
      discrepancies,
      totalMassVariance,
      timestamp: Date.now()
```

```
    };
  }
}
```

4 Verification and Conclusion

Unit tests within `tests/sprint_083.test.ts` validate correct discrepancy computation across matching and mismatched state maps. Furthermore, UML schema updates in `db/uml/sprint_083_schema.puml` document the validator relationships. Sub-Task A establishes the rigid interface foundation required for automated thermodynamic compensation monads within the Web of Life ecosystem.

References

[1] Web of Life Project. (2025). *Official Repository and Simulation Engine.* `https://github.com/pascalranoroarijaona/WebOfLife`